10.2 Security measures implemented

The AI interview assistant incorporates several security measures to protect user data and prevent system abuse. These measures are implemented in a middleware.js function to ensure that all incoming requests are validated and secured before they reach the application's core logic:

- *Authentication*—As mentioned before, we use Clerk.js to ensure that all users are authenticated before accessing the interview application. This helps control the application's use and stores user session data securely in Redis.
- *Cross-origin resource sharing (CORS) handling*—We use a middleware function to configure CORS and to allow requests from specific origins while blocking others. This prevents unauthorized access to the application's resources from malicious websites.
- *Rate limiting*—Implemented using Upstash Redis, the `rateLimit` function limits the number of requests a user can make within a specific time frame. This prevents abuse by malicious actors who might try to overload the server with excessive requests.
- *Security headers*—The `securityHeaders` middleware function sets several HTTP headers to enhance security.

Of course, these security measures only serve as a minimum requirement when implementing publicly available web applications. Try to maintain a proactive mindset when considering the security of your platform and stay informed about emerging threats and vulnerabilities.

10.3 Challenges during development

During the development of the AI interview assistant, I encountered several challenges and applied innovative solutions to overcome them.

10.3.1 State management considerations

Managing the state of the conversation, including the history of questions and answers, and maintaining context awareness for the AI interviewer proved difficult. We aim to enable users to seamlessly retrieve their session data for each past interview and quickly switch between threads without any problem.

To solve this, I opted to use Upstash Redis for storing session data, including the conversation history and user-specific settings. Redis's speed and efficiency make it ideal for managing real-time conversation data. However, this required establishing a simple ad hoc correlation between session IDs and session messages using a key-value database. While Redis is not inherently designed for complex relational models, we can make it work by creating and programmatically maintaining these straightforward pairings. Ideally, of course, you want to use SQL since it supports transactions on the persistence layer.